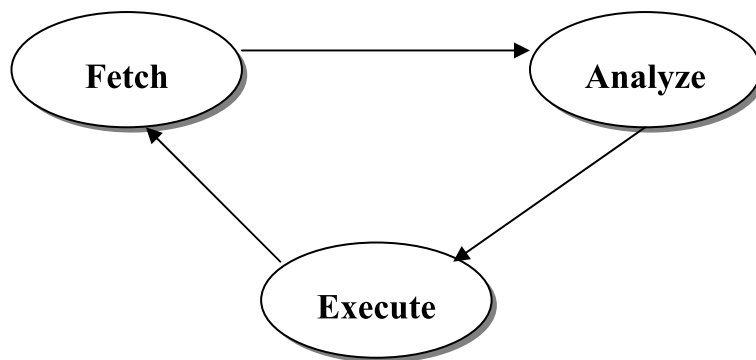


บทที่ 3

อินเตอร์พรีตเตอร์

อินเตอร์พรีตเตอร์เป็นโปรแกรมที่ทำการอ่านโปรแกรมหนึ่งๆ ที่เขียนขึ้นจากภาษาใดๆ ซึ่งเรียกว่าซอร์สโค้ด แล้วทำการแปลความหมายของโค้ดนั้นทีละหนึ่งชุดคำสั่ง หรือทีละหนึ่งการกระทำ เพื่อนำไปกระทำตามความหมายของคำสั่งนั้นๆ โดยการทำงานทุกๆ ไปของอินเตอร์พรีตเตอร์นั้นจะมีอยู่ด้วยกัน 3 ส่วนใหญ่ๆ ดังรูปต่อไปนี้



รูปที่ 3-1 แสดงส่วนประกอบหลักของอินเตอร์พรีตเตอร์

3.1 การเฟิชคำสั่ง (Fetch)

การเฟิชคำสั่งนั้น คือการอ่านข้อมูลจากซอร์สโค้ดเข้ามายังตัวโปรแกรมอินเตอร์พรีตเตอร์ โดยที่ส่วนของการเฟิชคำสั่งนั้นจะต้องมีการทำการเช็ค ว่าควรจะอ่านมาเท่าใดซึ่งในหลักการแล้ว ก็จะต้องทราบให้ได้ว่าบรรทัดที่เฟิชเข้ามานั้น ได้ข้อมูลมาครบหนึ่งคำสั่งหรือไม่ หากยังไม่ครบจำเป็นที่จะต้องนำส่วนที่เหลือออกมาให้ครบเป็น 1 ชุดคำสั่ง เช่น

$x = 10$

จากตัวอย่างจะเห็นว่าหากเราอ่านข้อมูลเข้ามาจากบรรทัดแรกมาเพียงบรรทัดเดียวนั้นก็ถือว่าเพียงพอแล้ว เนื่องจาก $x = 10$ นั้นถือเป็นคำสั่ง 1 คำสั่งแล้ว แต่ถ้าหากเป็นดังตัวอย่างข้างล่างนี้

if $x < 10$:

$y = 100$

เราจะต้องทราบให้ได้ว่า การเฟิร์สแค่บรรทัดเดียวนั้นไม่เพียงพอที่จะนำข้อมูลนั้นไปวิเคราะห์ได้ จำเป็นที่จะต้องทำการอ่านข้อมูลมาอีก 1 บรรทัดเพื่อให้ได้ครบเป็น 1 ชุดคำสั่ง

3.2 การวิเคราะห์ข้อมูล (Analyze)

การวิเคราะห์ข้อมูลนั้นจะเป็นการนำข้อมูลที่ได้นำมาทำการเฟิร์สเข้ามาทำการตรวจจับว่าค่าต่างๆ ที่นำเข้ามานั้นถูกต้องตามรูปแบบของภาษานั้นๆ หรือไม่ (Lexical Analysis) จากนั้นหากพบว่าค่าแต่ละค่าถูกต้องตามรูปแบบที่กำหนดไว้แล้วก็จะทำการนำค่าเหล่านั้นมามองเป็นรูปประโยคแล้วตรวจสอบว่าประโยคที่ได้จากค่าต่างๆ มารวมกันนั้นถูกต้องตามรูปประโยคของภาษานั้นๆ (Syntax Analysis) หรือไม่ และเมื่อพบว่าประโยคนั้นๆ ถูกต้องสมบูรณ์แล้วก็จะนำประโยคที่ได้ไปแปลความหมายว่าชุดคำสั่งนั้นเป็นการกระทำใดเพื่อนำไปสร้างโครงสร้างของคำสั่งให้อยู่ในรูปแบบของทรี (Tree) ซึ่งเรียกกันว่าพาร์ซทรี (Parse Tree) เพื่อส่งต่อให้ส่วนการกระทำตามคำสั่งนั้นๆ อีกที

3.2.1 การตรวจจับคำ (Lexical Analysis)

เป็นการตรวจจับกลุ่มของสตริงค์ (String) เพื่อให้ทราบว่าสตริงค์ต่างๆ เหล่านั้นเป็นกลุ่มคำที่มีอยู่ในภาษาที่เราต้องการจะทำการอินเตอร์พรีตหรือไม่ (ซึ่งในที่นี้จะหมายถึงภาษาไพธอน) โดยกลุ่มคำดังกล่าวนั้นจะต้องมีการกำหนดขึ้นมาโดยการใช้เรกกูลาร์เอ็กซ์เพรสชัน (Regular Expression) สำหรับกำหนดรูปแบบของตัวกลุ่มคำ

เรกกูลาร์เอ็กซ์เพรสชัน คือ การกำหนดกลุ่มของสตริงค์โดยใช้สัญลักษณ์ต่างๆ ที่เป็นมาตรฐานในการกำหนดความหมายต่างๆ ให้กับสตริงค์ โดยจะมีอยู่หลักๆ ดังต่อไปนี้

' '	เลือกเอาไอเท็มตัวใดตัวหนึ่งที่ขึ้นกลางด้วยสัญลักษณ์นี้
'*'	กำหนดว่าไอเท็มนั้นจะมีได้ตั้งแต่ 0 ตัวถึงเท่าไรก็ได้
'+'	กำหนดว่าไอเท็มนั้นจะมีได้ตั้งแต่ 1 ตัวขึ้นไป
'?'	กำหนดว่าไอเท็มนั้นจะมีได้ตั้งแต่ 0 หรือ 1 ตัว
'(' และ ')'	กำหนดกลุ่มไอเท็ม

ตัวอย่าง เรกกูลาร์เอ็กซ์เพรสชันสำหรับกลุ่มคำ (Token) ของภาษาไพธอนจะออกมามีลักษณะดังต่อไปนี้

ชื่อ	เรกูลาร์เอ็กซ์เพรสชัน
identifier	$(\{\text{letter}\} _ ")(\{\text{letter}\} \{\text{digit}\} _ ")^*$
letter	$(\{\text{lowercase}\} \{\text{uppercase}\})$
lowercase	[a-z]
uppercase	[A-Z]
digit	[0-9]
integer	$(\{\text{decimalinteger}\} \{\text{octinteger}\} \{\text{hexinteger}\})$
longinteger	$\{\text{integer}\} [\text{L}]$
intpart	$\{\text{digit}\}^+$
fraction	$"." \{\text{digit}\}^+$
exponent	$[\text{eE}] [+ -]? \{\text{digit}\}^+$
decimalinteger	$\{\text{nonzerodigit}\} \{\text{digit}\}^* 0$
octinteger	$0 \{\text{octdigit}\}^+$
hexinteger	$0 [\text{xX}] \{\text{hexdigit}\}^+$
nonzerodigit	[1-9]
octdigit	[0-7]
hexdigit	$(\{\text{digit}\} [\text{a-f}] [\text{A-F}])$

ตารางที่ 3-1 ตารางตัวอย่างเรกูลาร์เอ็กซ์เพรสชันของภาษาไพธอน

3.3 การตรวจสอบประโยค (Syntax Analysis)

เป็นการตรวจสอบประโยคที่ได้มาจากกลุ่มคำที่ถูกต้องว่าเป็นประโยคที่ถูกต้องตามรูปแบบที่กำหนดไว้หรือไม่ ซึ่งในการที่จะกำหนดรูปแบบของประโยคต่างๆ ขึ้นมาได้นั้นจะต้องมีการกำหนดขึ้นโดยใช้บีเอ็นเอฟ (BNF : Backus-Naur Form)

บีเอ็นเอฟ นั้นจะเป็นการกำหนดกฎหรือตั้งกฎ (rule) ของประโยคต่างๆ ของภาษาใดภาษาหนึ่งที่ต้องการสร้างประโยค (ซึ่งในที่นี้หมายถึงภาษาไพธอน) โดยลักษณะของ บีเอ็นเอฟจะมีลักษณะดังนี้

$\langle \text{non terminal} \rangle \rightarrow \langle \text{non terminal} \rangle \text{ หรือ } \langle \text{terminal} \rangle | \dots | \langle \text{non terminal} \rangle \text{ หรือ } \langle \text{terminal} \rangle$

3.3.1 non terminal

non terminal คือ สัญลักษณ์ที่ใช้แสดงถึงจุดที่ยังไม่สิ้นสุดของประโยคแต่สามารถที่จะขยายออกไปอีกได้โดยใช้กฎของ บีเอ็นเอฟ ตัวอื่นๆ ต่อไป เช่น

$$\begin{array}{lcl} A & \rightarrow & B \mid C \\ B & \rightarrow & "x" \\ C & \rightarrow & "y" \end{array}$$

จากตัวอย่างข้างต้น ถ้าสมมติให้ ตัวอักษรตัวหนาหมายถึง non terminal ก็จะเห็นว่าจาก บีเอ็นเอฟ บรรทัดแรกนั้น สามารถขยายต่อไปยัง บีเอ็นเอฟบรรทัดที่สองและบรรทัดที่สามได้อีกจนกว่าจะไปเจอส่วนที่เป็น terminal ก็จะไม่สามารถขยายต่อได้อีกนั่นเอง

3.3.2 terminal

terminal คือ จุดสิ้นสุดของกฎในบีเอ็นเอฟ ซึ่งอาจหมายถึงคำใดๆ หรือตัวอักษรใดๆ รวมทั้งกลุ่มคำที่ได้มาจากการกำหนดเรกูลาร์เอ็กซ์เพรสชันในขั้นตอนตรวจสอบจับกลุ่มคำก็ได้

ตัวอย่าง บีเอ็นเอฟ สำหรับภาษาไพธอนจะมีลักษณะดังต่อไปนี้

ชื่อ	บีเอ็นเอฟ
primary	atom target
atom	literal enclosure
target	IDENTIFIER subscription
literal	INTEGER STRING LONGINTEGER
sSubscription	primary '[' expression_list ']'
enclosure	(' expression_list ') '[' expression_list ']'

ตารางที่ 3-2 ตารางตัวอย่างบีเอ็นเอฟของภาษาไพธอน

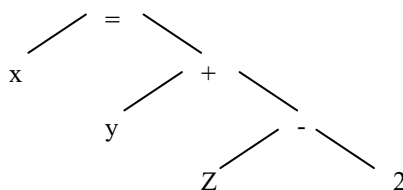
3.4 พาร์ซทรี (Parsetree)

พาร์ซทรี นั้นจะเป็นโครงสร้างของประโยค ที่ผ่านการตรวจสอบจากขั้นตอนที่ผ่านมาเรียบร้อยแล้ว โดยเราจะทำการแยกกลุ่มคำแต่ละตัวของประโยคออกมาสร้างเป็นลักษณะของทรี ซึ่งโครงสร้างดังกล่าวนี้เราจะเรียกว่า Abstract Syntax Tree (AST)

โดยโครงสร้างของคำสั่งแต่ละคำสั่งนั้นก็อาจจะมีลักษณะที่แตกต่างกันออกไปตามแต่ลักษณะของคำสั่งโดยจะพิจารณาเอากลุ่มคำที่แสดงถึงการกระทำเป็นราก (root) ของทรีส่วนลูกทางด้านซ้าย (left child) และลูกทางด้านขวา (right child) นั่นก็จะเป็นส่วนของกลุ่มคำที่เป็นตัวถูกกระทำ โดยการสร้างพาร์ซทรีนั้นจะทำการตรวจสอบและสร้างจากประโยคซึ่งจะมองจากซ้ายไปขวา และความสำคัญของการกระทำเป็นหลัก

ตัวอย่าง พาร์ซทรีของคำสั่งทั่วไปในภาษาไพธอน เป็นดังต่อไปนี้

fetch input : $x = y + Z - 2$

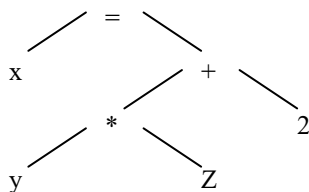


รูปที่ 3-2 แสดงพาร์ซทรีของคำสั่งทั่วไปของภาษาไพธอน

จากรูปจะเห็นได้ว่าการสร้างพาร์ซทรีต้นนี้จากซ้ายไปขวาโดยที่ตัวกระทำตัวแรกที่เรพบคือ เครื่องหมายเท่ากับ (=) จึงนำไปเป็นรากของทรีจากนั้นทางด้านซ้ายของเครื่องหมายเท่ากับก็คือ x เราก็นำไปเป็นลูกทางซ้ายของต้นไม้อีก จากนั้นทางด้านขวาของเครื่องหมายเท่ากับนั่นคือ $y + Z - 2$ ซึ่งก็จะต้องเริ่มดูตัวกระทำอีกว่าเป็นตัวอะไรซึ่งก็คือเครื่องหมายบวก (+) จึงนำมาเป็นลูกทางขวาของทรีต้นที่มีรากเป็นเครื่องหมายเท่ากับ แต่เครื่องหมายบวกเองก็จะเป็นรากของทรีต้นใหม่ที่สร้างขึ้นซึ่งก็จะมีลูกด้านซ้ายเป็น y และมีลูกทางด้านขวาเป็น $Z - 2$ ซึ่งก็จะต้องสร้างเป็นทรีต่อไปจนกว่าจะไม่สามารถสร้างต่อได้

จะสังเกตเห็นได้จากตัวอย่างข้างต้นนั้น เครื่องหมายบวก และลบมีนัยสำคัญของการกระทำเท่ากันจึงมีการสร้างทรีจากซ้ายไปขวาได้อย่างปกติแต่หากว่าการกระทำต่างๆ นั้นมีนัยสำคัญไม่เท่ากันเราก็จะต้องคำนึงถึงจุดนี้ด้วยดังตัวอย่างต่อไปนี้

fetch input : $x = y * Z + 2$



รูปที่ 3-3 แสดงพาร์ซทรีของคำสั่งที่มีนัยสำคัญของการกระทำไม่เท่ากัน

จากรูปที่ 3-3 จะเป็นได้ว่าเครื่องหมายคูณ (*) กับเครื่องหมายบวกนั้น จะมีนัยสำคัญของการกระทำไม่เท่ากัน โดยที่เครื่องหมายคูณนั้นจะมีนัยสำคัญมากกว่าเครื่องหมายบวกจะนั้นที่สร้างขึ้นนั้นก็จะต้องเอา $y * Z$ ไว้ลึกสุด หรือหมายถึงจะต้องมีการประมวลผลก่อน จากนั้นจึงทำการนำค่าที่ได้มาบวกด้วย 2 นั่นเอง

3.5 การกระทำตามคำสั่ง (Execution)

เมื่อผ่านขั้นตอนของการพาร์ซทรีมาแล้ว เราจะได้โครงสร้างของคำสั่งนั้นๆ ซึ่งจะนำมากระทำตามคำสั่งนั้นๆ โดยการกระทำจากทรีโหนดที่ลึกที่สุด (leave node) ของทรีก่อน แล้วจึงคืนค่ากลับ (return) ไปให้กับโหนดที่เป็นราก (root node) เพื่อให้ส่วนของทรีที่อยู่สูงกว่ากระทำตามคำสั่งต่อไป จนถึงจุดสูงสุดของทรีนั่นเอง

โดยหากเราลองมาพิจารณาจากรูปที่ 3-1 ก็จะได้การกระทำตามคำสั่งดังต่อไปนี้

1. พิจารณาจากทรีโหนดที่ลึกที่สุด ซึ่งก็คือทรีของ $Z - 2$ ซึ่งเราก็จะทำการลบค่า Z ด้วย 2 แล้วคืนค่าผลลัพธ์กลับขึ้นไปยังตำแหน่งรากของทรี สมมติให้ค่าของ Z เท่ากับ 10 เราจะได้ค่า 8 คืนกลับขึ้นไป
2. พิจารณาจากทรีที่ถัดขึ้นไป ซึ่งก็คือทรีของ $y + \text{<ผลลัพธ์ของทรีต้นที่ผ่านมา>}$ ซึ่งเราก็จะทำการบวกค่า y ด้วย ผลลัพธ์ที่ได้จากทรีต้นที่ผ่านมาซึ่งคือ 8 แล้วคืนค่าผลลัพธ์กลับขึ้นไปยังตำแหน่งรากของทรี สมมติให้ค่าของ y เท่ากับ 10 เราจะได้ค่า 18 คืนกลับขึ้นไป
3. พิจารณาจากทรีที่ถัดขึ้นไปอีก ซึ่งก็คือทรีของ $x = \text{<ผลลัพธ์ของทรีต้นที่ผ่านมา>}$ ซึ่งเราก็จะทำการกำหนดค่าให้กับ x ด้วยผลลัพธ์ที่ได้จากทรีต้นที่ผ่านมาซึ่งคือ 18 แล้วคืนค่าผลลัพธ์กลับขึ้นไปยังตำแหน่งรากของทรี ซึ่งในที่นี้ไม่มีทรีชั้นต่อไปแล้วจึงเป็นอันเสร็จการกระทำตามคำสั่งนี้